

完成第一次挑战的作业感受

(1) 简述我是如何完成第一次挑战的：

首先，因为我的电脑是 AMD 显卡，我最初尝试在 Windows 里用 WSL2+Ubuntu 搭建环境。想要硬上，在之后的操作中就遇到了这些问题：WSL 安装超时、下载到了 18.8KB 的假安装包，好不容易装好后，又发现 Ubuntu 自带的 Python 3.14 太新，报错找不到 Rust 编译器，根本无法安装 SGLang。

在本地用 CPU 强跑报错缺失 vllm 模块后，我果断尝试其他实现方式，转战云 GPU 平台。在优云智算上抢到 4090 后，又遭遇了网页卡死、云镜像缺 C++ 编译器 (cc1plus)、PyTorch 与 TorchAudio 的 CUDA 版本冲突等连环坑。我靠着通过**看日志+依赖 AI 精准定位**，一步步补装 **build-essential**、**卸载冲突的 torchaudio**，最终成功跑通了 SGLang，拿到了完美的 JSON 响应。为了解决本地无法稳定复现 Mooncake trace 的问题，我**查阅了 Poisson 到达过程的底层逻辑，借助 AI 辅助生成了 Python 模拟脚本，并在本地环境手动激活、调试和运行**，最终成功打印出了包含 tokens、status、latency 和 TTFT 的完整 workload 日志。

(2) 我觉得哪部分工作最困难，以及我是如何克服的：

最困难的是**云平台的“黑盒”排错与资源受限**。本地报错还能慢慢搜，云平台报错，机器按小时计费、随时可能因为抢占式实例被回收。

克服的方式是**“精准定位与及时止损”**。比如遇到 RuntimeError 时，我没有盲目重装环境，而是仔细阅读最后几行报错，发现是 TorchAudio 和 PyTorch 的 CUDA 版本不一致，卸载它后立刻解决。当云端环境实在折腾不完

时,我没有继续死磕,而是向 AI 请教了如何构造模拟 workload 的代码逻辑,拿到基础代码框架后,在本地亲自完成了环境激活、语法调试和多次报错修复,最终成功运行。这段模拟代码基于指数分布实现了 Poisson 到达过程,成功打印出了包含 tokens、status、latency 和 TTFT 的完整日志。

这让我深刻认识到:科研中,懂得如何排查问题、善用现代工具(包括 AI)并独立验证结果,比盲目努力更重要。

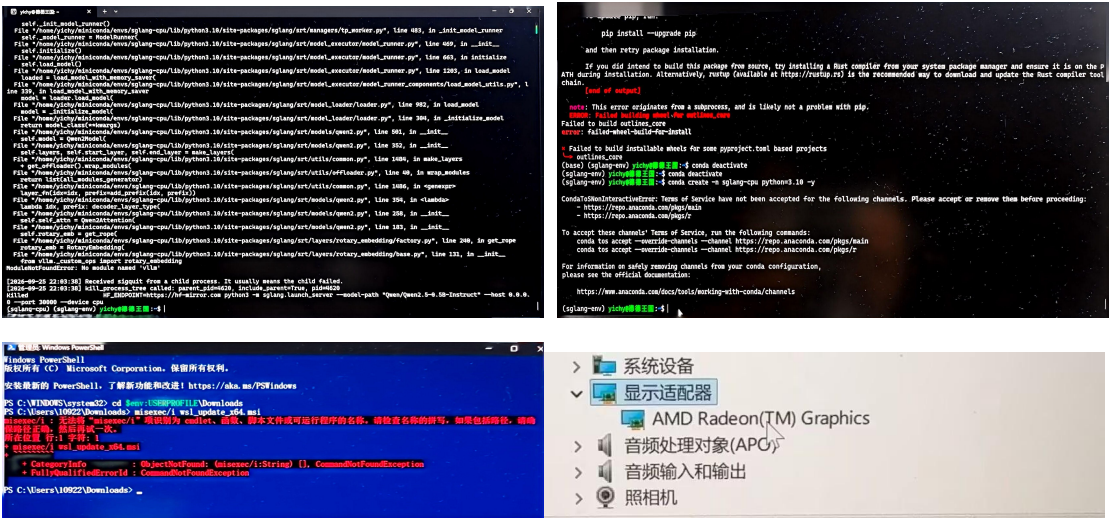


图:本地环境排错过程(依次解决了WSL安装、Python3.14 Rust编译器缺失、以及AMD显卡导致得CPU模式缺失vllm等问题)

(3) 从 0 到 1 的科研工作,需要哪些方面的研究,对我今后有什么启发:

经过这次挑战我发现科研比我想象中的难的多得多。真正的从 0 到 1,前期的环境搭建、系统架构和工程实现占据了极大比重。不懂底层依赖(如 CUDA、Conda 环境隔离、Linux 系统调用),算法再好也跑不起来。

在未来的科研工作中,我将不再轻视“跑通代码”这一步,而是主动积累推理优化(如 KV Cache 管理)和系统架构方面的工程知识。同时,我也学会了及时止损与方案切换(比如从本地 CPU 到云端 GPU,再回到本地脚本模拟),

以及善用现代 AI 工具辅助排查。

经过这次挑战，我深刻认识到了系统底层工程对于大模型推理的重要性。我非常渴望能在实验室中进一步学习推理优化技术，也做好了随时面对硬核难题并死磕到底的准备。